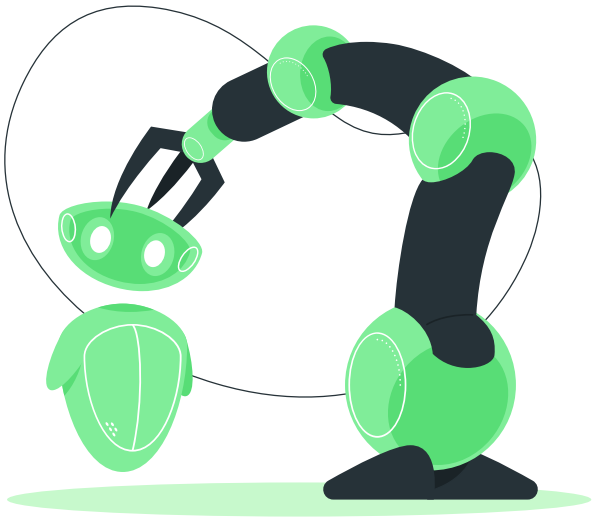
Robotics
CS 3015

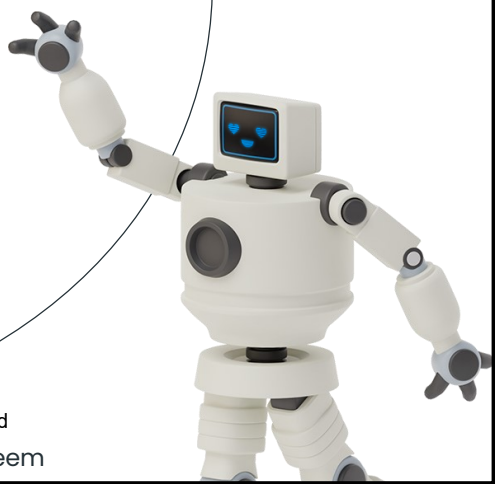
Umarah Qaseem



1

14

- Arduino
- Hardware
- Anatomy
- Basic Operation
- Circuit Basics
- Schematics
- Digital Signal
- Analog Signal
- Sensor and Actuators
- Serial Communication Protocols
- Memory
- Internet of things IOT
- Arduino API
- Sketch, structure
- IDE, Software tools
- Arduino Cloud
- Arduino CLI
- Arduino Language (Complete tutorial if-else, loops, serial monitor, arithmetic, logical relational operators, compound operators)



2

Arduino



Robotics – Umarah Qaseem

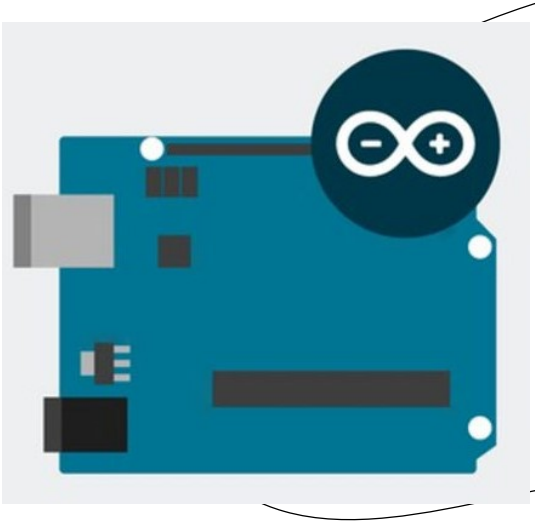
4

Microcontroller or Microprocessor?



Robotics – Umarah Qaseem

5



Hardware

6

Basics

- Started in 2005
- Over the years, Arduino has released hundreds of hardware designs in many shapes and forms.

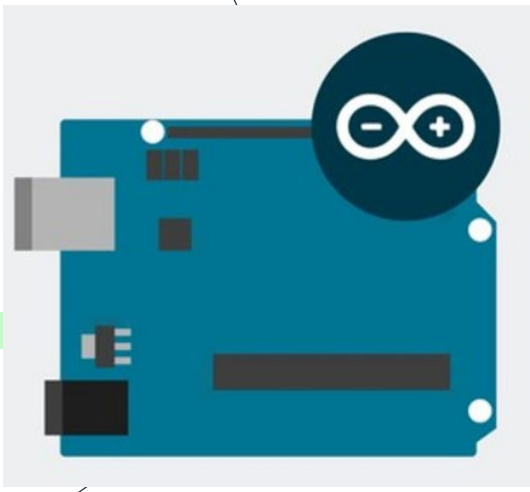
7

Hardware

- While all Arduino boards differ from each other, there are several key components that can be found on practically any Arduino.
- Let's take a look at the image in next slide:

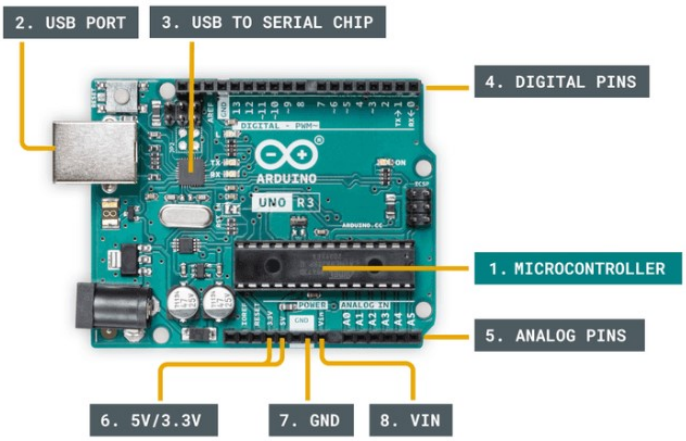
8

Anatomy



9

Anatomy of Board



10

Key components

1. **Microcontroller** - this is the brain of an Arduino, and is the component that we load programs into. Think of it as a tiny computer, designed to execute only a specific number of things.
2. **USB port** - used to connect your Arduino board to a computer.
3. **USB to Serial chip** - the USB to Serial is an important component, as it helps translating data that comes from e.g. a computer to the on-board microcontroller. This is what makes it possible to program the Arduino board from your computer.

11

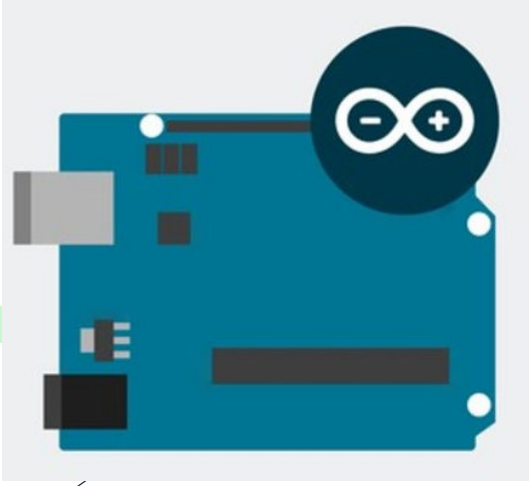
Key components

4. **Digital pins** - pins that use digital logic (0,1 or LOW/HIGH). Commonly used for switches and to turn on/off an LED.
5. **Analog pins** - pins that can read analog values in a 10 bit resolution (0-1023).
6. **5V / 3.3V pins-** these pins are used to power external components.
7. **GND** - also known as ground, negative or simply -, is used to complete a circuit, where the electrical level is at 0 volt.
8. **VIN** - stands for Voltage In, where you can connect external power supplies.

Robotics – Umarah Qaseem

12

Basic Operation



Robotics – Umarah Qaseem

13

Basic Operation

- Most Arduino boards are designed to have a single program running on the microcontroller.
- This program can be designed to perform one single action, such as blinking an LED.
- It can also be designed to execute hundreds of actions in a cycle.
- The program that is loaded to the microcontroller will start execution as soon as it is powered.
- Every program has a function called "loop".

14

What can you do with it?

- Example:
- Inside the loop function, you can
  - Read a sensor.
  - Turn on a light.
  - Check whether a condition is met.
  - All of the above.

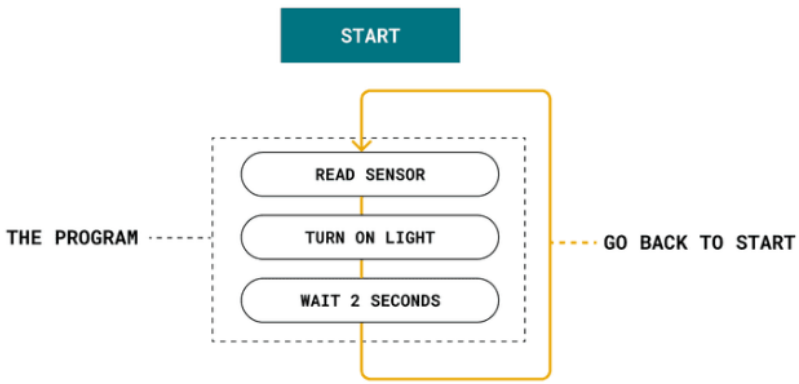
15

Speed of the program

- The speed of a program is incredibly fast, unless we tell it to slow down.
- It depends on the size of the program and how long it takes for the microcontroller to execute it, but it is generally in **microseconds (one millionth of a second)**.
- We can use sleep()!

16

Basic working



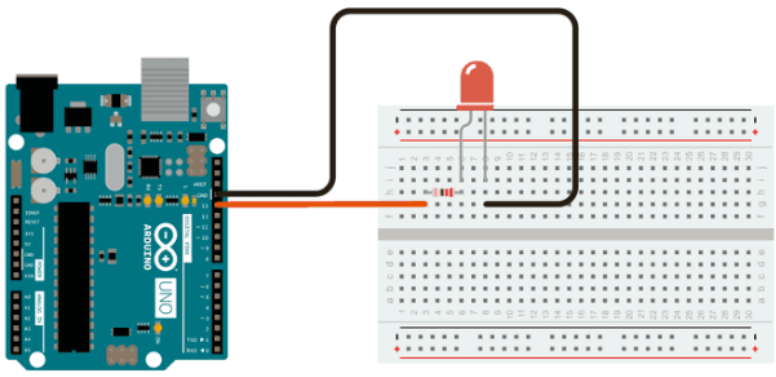
17

Circuit Basics

- Circuits consist of at least one **active electronic component**, and a **conductive material**, such as wires, so that current can pass through
- When working with an Arduino, you will in most cases build a circuit for your project.
- A simple example: **LED circuit**
- A wire is connected from a pin on the Arduino, to an **LED via a resistor** (to protect the LED from high current), and finally to the **ground pin** (GND).
- When the pin is set to a **HIGH state**, the microcontroller on the Arduino board will allow an electric current to flow through the circuit, which turns on the LED.
- When the pin is set to a **LOW state**, the LED will turn off, as an electric current is not flowing through the circuit.

18

Circuit Basics – Example



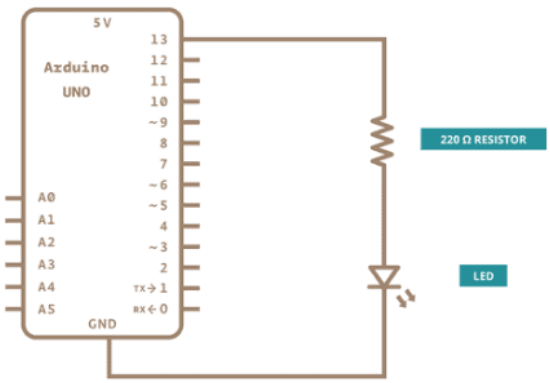
19

Schematic Diagram

- Circuits are typically represented as **schematics**, which are the blueprints for your circuit.
- The image on the next slide shows the schematic's representation of the same circuit shown in the previous slide.

20

Schematic Diagram



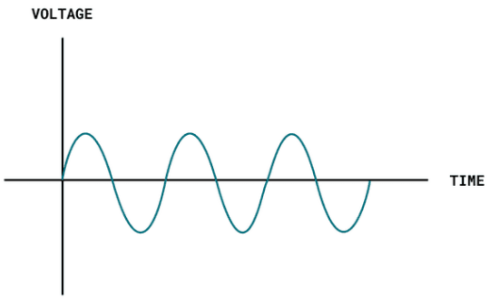
21

Electronic Signals

- All communication between electronic components are facilitated by **electronic signals.**
- There are two main types of electronic signals:
  - **Analog**
  - **digital**.

22

Analog signal

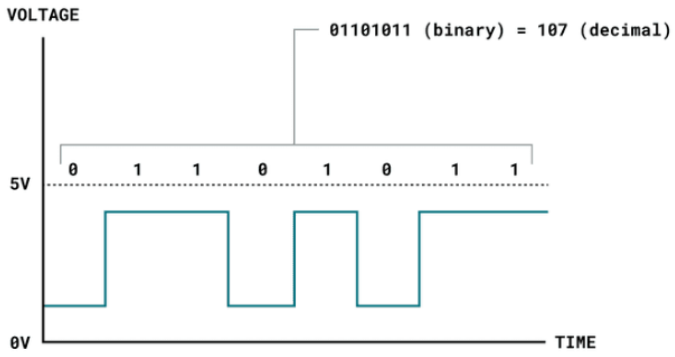


23

Analog signal

- An analog signal is generally bound to a range. In an Arduino, that range is typically 0-5V, or 0-3.3V.
- For example: If we use a potentiometer (an analog component used to change the resistance of a circuit), we can manually adjust this range (0-5V). In the program, this is represented in a range of 0-1023, which is a 10-bit resolution.
- If we write an analog signal using Pulse-Width Modulation (PWM), we can use a range between 0-255, as we are using an 8-bit resolution.

Digital Signal



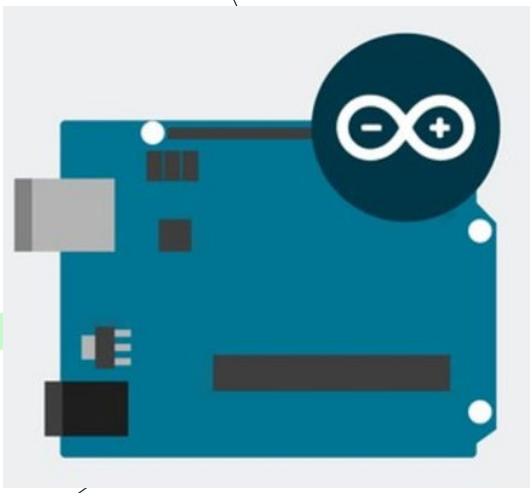
Digital Signal

- Only two binary states (0 or 1) that are read as high or low states in the program.
- useful to read button states, or to turn something on or off.
- Digital signals seem very basic (0,1) but are way more advanced.
- For example, we can create a sequence by sending a high or low state rapidly a number of times. This is known as a **binary sequence** or a **bitstream**. (example on next slide)

Bit Stream

- Example:
- 101101
- 101110001110011
- In decimal are
- 45
- 23667
- This is a clever way of sending large amounts of data from one point to the other, by rapidly sending high & low signals.
- In order to interpret the data from the signals, we use **Serial Communication Protocols.**

Sensors & Actuators



Robotics – Umarah Qaseem

28

What is A sensor?

- A sensor is used to sense its environment. This means it records a physical parameter, e.g. temperature, and converts it into an electronic signal.
- Sensors can also take the form of just a simple button: when a state changes (we pressed a button), the electronic signal is switched from low to high (0 to 1).

Robotics – Umarah Qaseem

29

Analog Sensors

- There are many types of sensors, and several ways of recording data from them. The easiest to use is an analog sensor, where we communicate a range of values through altering the voltage input fed into an Arduino analog pin (usually between 0-5 volts). This simply gives you a range between 0-1023 (a 10-bit resolution).

30

Digital Sensors

- Digital sensors are a bit more advanced, depending on the type.
- They rely on Serial Communication Protocols to send the data accordingly and requires a bit more effort to translate the data.
- As mentioned in previous slides, data is sent using a binary sequence (e.g. 101101 is 45), and this needs to be addressed and configured on a software level. Luckily, a lot of sensors are accompanied by software libraries, which makes it a lot easier to read.

- In many cases using a library, all we need is just one line of code:

```
sensorValue = sensor.read();
```


31

What Is an Actuator?

- An actuator, in simple terms, is used to actuate or change a physical state. Some examples are:

- A light (such as an LED).
- A motor.
- A switch.
- Actuators converts electric signals into e.g. radiant energy (light) or mechanical energy (movement).

32

Actuator

- How actuators are controlled really depends on what type of component we have.
- The simplest way is to simply turn something on/off, while more advanced is controlling the amount of voltage a component receives (i.e. the speed of a motor).
- To control actuators, it is common to use **digitalWrite**() and **analogWrite**().

- Example:
- digitalWrite(LED, HIGH); //turn on an LED
- digitalWrite(LED, LOW); //turn off an LED

- analogWrite(motor, 255); //set a motor to maximum capacity
- analogWrite(motor, 25); //set a motor to 10% of its capacity

33

Input & Output

- Sensors and actuators, are typically referred to as **inputs and outputs**.
- When we write a program, it is common to construct conditionals that checks the state of a sensor and decides whether it should actuate something.
- A basic example of this is a **button** and an **LED**.
- We can write a conditional that checks if a button is pressed, turn on the LED, and turn it off if the button is not pressed.

34

Example Arduino program

```
//read and store the button state (0 or 1)
int buttonState = digitalRead(buttonPin);

//check if state is high (button is pressed)
if(buttonState == HIGH){
   digitalWrite(LED, HIGH);    //turn on LED
}
else {
   digitalWrite(LED, LOW);     //turn off LED
}
```


35

Serial Communication Protocols

- There are several serial communication protocols that uses the aforementioned digital signals to send data. The most common are **UART, SPI & I²C**. The UART protocol is among other things, used to send data between a computer and Arduino board, such as uploading a new program, or reading data directly from an Arduino.
- The SPI and I²C protocols are used for communication between both internal and external components. The communication is handled by something called a **serial bus**, which is attached to a specific pin on the Arduino.
- Using the I²C protocol, we can connect several sensors on the same pin and retrieve the data accurately. Each device has an address that we need to specify in the program, which we use when making data requests.

36

Memory

- The "standard" Arduino typically has two memories: SRAM and Flash memory.
- The **SRAM** (Static Random-Access Memory) is used to for example store the value of a variable (such as the state of a Boolean). When powered off, this memory resets.
- The **Flash memory** is primarily used to store the main program, or the instructions for the microcontroller. This memory is not erased when powered off so that the instructions for the microcontroller are executed as soon as the board is powered.
- How much memory is available on an Arduino varies from board to board.
- The **Arduino UNO** has a 32kB flash / 2kB SRAM
- **Nano 33 IoT** has 256kB flash / 32kB SRAM.
- You can find this information in the product's documentation.

37

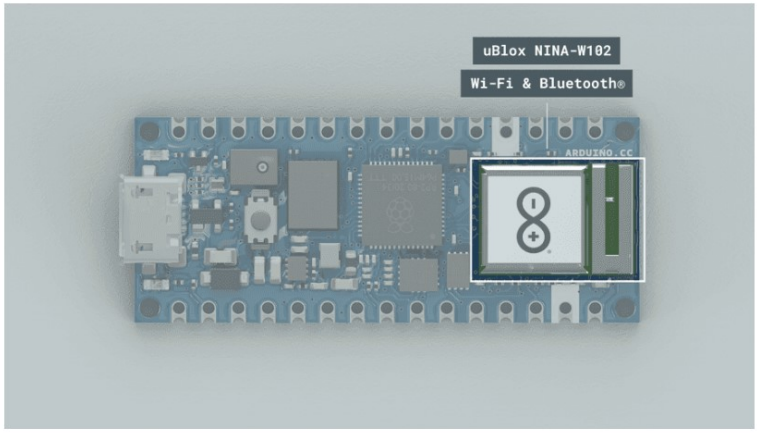
Internet of Things IOT

- Most modern Arduino boards now come equipped with a radio module, designed to communicate wirelessly.
- There are several different ones:
- Wi-Fi,
- Bluetooth®,
- LoRa®,
- GSM,
- NB-IoT and more.
- Each are designed to communicate using the various technologies available on the market.

Internet of Things IOT

- The most popular and inexpensive modules are the Wi-Fi & Bluetooth® modules.
- The Wi-Fi modules allow your board to connect to routers, and to request and send data over the Internet. In a way, it works the same as your computer when requesting various types of data over the Internet, just in a smaller scale.
- Bluetooth® is used to communicate with nearby devices and is really useful for maintaining a fast and reliable connection. For example, in real-life applications, Bluetooth® technology for example used in wireless headphones & speakers.
- Similarly to serial protocols, radio modules use their own set of protocols to communicate, such as HTTP, MQTT and UPD.

Internet of Things IOT

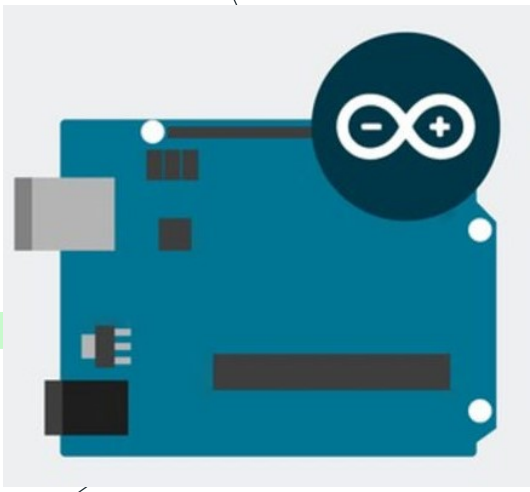


The u-blox NINA-W102 Wi-Fi / Bluetooth® module on the Nano RP2040 Connect board.

Robotics – Umarah Qaseem

40

Arduino API



Robotics – Umarah Qaseem

41

Arduino API

- The Arduino API can be divided into three main parts: functions, variables and structure:

- Functions
- Variables
- Structure

42

Arduino API

- **Functions**: for controlling the Arduino board and performing computations. E.g digitalWrite()
- **Variables**: the Arduino constants, data types and conversions.
- **Structure**: the elements of the Arduino (C++) code, such as
- **sketch** (loop(), setup())
- **control structure** (if, else, while, for)
- **arithmetic operators** (multiplication, addition, subtraction)
- **comparison operators** (== (equal to), != (not equal to), > (greater than))
- The Arduino API can be described as a simplification of the C++ programming language, with a lot of additions for controlling the Arduino hardware.

43

Program Structure

- The absolute minimum requirement of an Arduino program is the use of two functions: void setup() and void loop().
- void **setup**() - this function executes only once, when the Arduino is powered on. Here we define things such as the mode of a pin (input or output), the baud rate of serial communication or the initialization of a library.
- void **loop**() - this is where we write the code that we want to execute over and over again, such as turning on/off a lamp based on an input, or to conduct a sensor reading every X second.
- The above functions are always required in an Arduino sketch, but you can add several more functions, which is useful for longer programs.

Sketch

- In the Arduino project, a program is referred to as a "sketch".
- A sketch is a file that you write your program inside.
- It has the .ino extension.
- It is always stored in a folder of the same name.
- The folder can include other files, such as a header file, that can be included in your sketch.

Example Sketch

```
/*
This is a comment at the top of a program,
it will not be recognized as code. Very good
to add an explanation of what your code does
here.

This sketch shows how to read a value from a
sensor connected to pin A1, print it out in
the Serial Monitor, and turn on an LED connected
to pin number 2 if a conditional is met.
*/

int sensorPin = A1; //define pin A1 (analog pin)
int ledPin = 2; //define pin 2 (digital pin)
int sensorValue; //create variable for storing
readings

//void setup is for configurations on start up
void setup() {
  Serial.begin(9600); //initialize serial
communication
  pinMode(ledPin, OUTPUT); //define ledPin as
an output
}

void loop() {
  sensorValue = analogRead(sensorPin); // do
a sensor reading

  Serial.print("Sensor value is: "); //print a
message to the serial monitor
  Serial.println(sensorValue); //print the value
to the serial monitor

  //check if sensorValue is below 200
  if(sensorValue < 200) {
    digitalWrite(ledPin, HIGH); //if it is, turn on
the LED on pin 2.
  }
  //if sensorValue is above 200, turn off the
LED
  else{
    digitalWrite(ledPin, LOW);
  }
}
```


Arduino Software Tools

- Currently, there are three Arduino IDEs available:

- Arduino IDE 1.8.x (classic)
- Arduino IDE 2 (new)
- Arduino Cloud Editor (online)

Arduino IDE

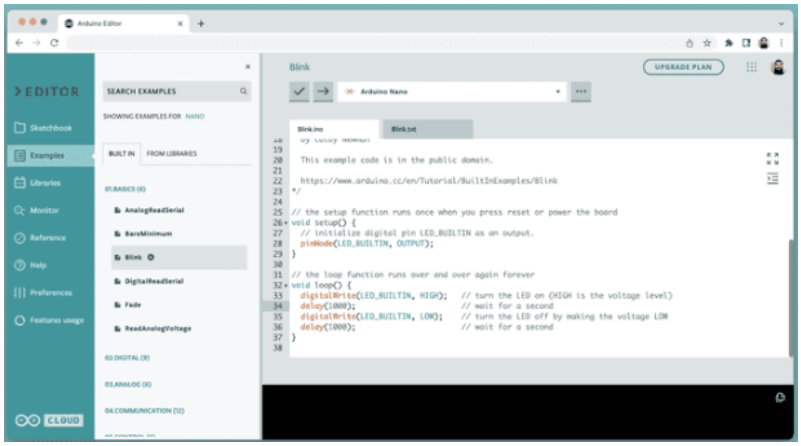


A Typical Workflow

To upload code to an Arduino board using the IDE, typical procedure is:

- **1. Install your board** - this means installing the right "package" for your board. Without the package, you can simply not use your board. Installing is done directly in the IDE, and is a quick and easy operation.
- **2. Create a new sketch** - a sketch is your main program file. Here we write a set of instructions we want to execute on the microcontroller.
- **3. Compile your sketch** - the code we write is not exactly how it looks like when uploaded to our Arduino: compiling code means that we check it for errors, and convert it into a binary file (1s and 0s). If something fails, you will get this in the error console.
- **4. Upload your sketch** - once the compilation is successful, the code can be uploaded to your board. In this step, we connect the board to the computer physically, and select the right serial port.
- **5. Serial Monitor (optional)** - for most Arduino projects, it is important to know what's going on on your board. The Serial Monitor tool available in all IDEs allow for data to be sent from your board to your computer.

Cloud Editor



Cloud Editor

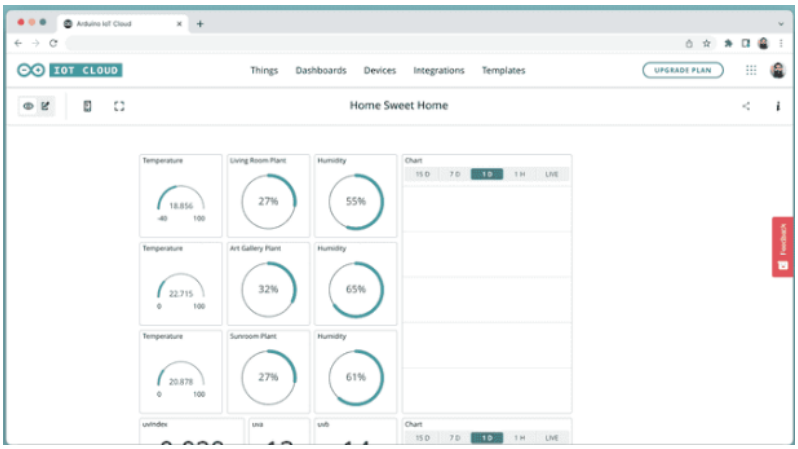
- The Arduino Cloud Editor is an online IDE, part of the Arduino Cloud suite.
- Similar in function,
- completely web based,
- with online storage among other features.
- To use the Cloud Editor, you will need to register an Arduino account.

Arduino Cloud

- The Arduino Cloud allows you to configure, program and control/monitor your devices - all in one web-based application.
- With the use of things, or your "digital twin", you can control and monitor variables directly from dashboards.
- The service also supports webhooks and integrations with other services, such as Amazon Alexa.
- The Cloud is made for anyone to use, and it does not require much previous experience to get started.

52

Arduino Cloud



53

Arduino CLI

- The Arduino CLI is a command line tool that can be used to compile and upload code to your board.
- It has no visual UI, but is very useful for automation.
- It is designed for more advanced users.
- A proper use of the CLI can speed up your development time by far, as any operation is executed much faster than in the regular IDE.

54

Arduino CLI



```
Usage:
  arduino-cli [command]

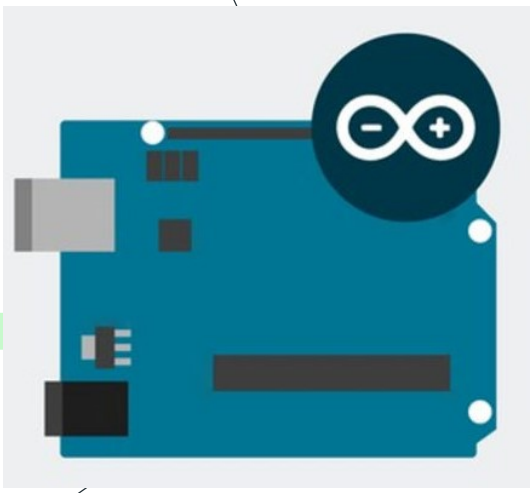
Examples:
  arduino-cli <command> [flags...]

Available Commands:
  board           Arduino board commands.
  burn-bootloader Upload the bootloader.
  cache           Arduino cache commands.
  compile         Compiles Arduino sketches.
  completion      Generates completion scripts
  config          Arduino configuration commands.
  core            Arduino core operations.
  daemon          Run as a daemon on port: 50051
  debug           Debug Arduino sketches.
  help            Help about any command
  lib             Arduino commands about libraries.
  monitor         Open a communication port with a board.
  outdated        Lists cores and libraries that can be upgraded
  sketch          Arduino CLI sketch commands.
  update          Updates the index of cores and libraries
  upgrade         Upgrades installed cores and libraries.
  upload          Upload Arduino sketches.
  version         Shows version number of Arduino CLI.

Flags:
      --additional-urls strings   Comma-separated list of additional URLs for the Boards Manager.
      --config-file string        The custom config file (if not specified the default will be used).
      --format string             The output format for the logs, can be: text, json, jsonmini, yaml (default "text"
```


55

Arduino Language



56

setup()

- The setup() function is where you make program configurations.

```
void setup() {
    //program configurations here
}
```


57

loop()

- The loop() function is where your main program is stored. It will run as long as your board is powered.

- void loop() {
- //main program here
- }

58

delay()

- The delay() function pauses the program for a set number of milliseconds.

- Example: The classic blink sequence:
- void loop() {
- digitalWrite(LED, HIGH); //turn on an LED
- delay(1000); //as program is paused, with the LED on
- digitalWrite(LED, LOW); //program is unpaused, and the LED is turned off
- delay(1000); //program is paused, with the LED off
- }

59

millis()

- The delay() function is an incredibly useful function, and you will find it in almost all examples. But, for efficiency of the code, it is not the best option, as it prevents the Arduino from doing anything for the duration of the delay.
- For this, we can use the millis() function.
- The millis() function is a bit more advanced, but an incredibly resourceful function. It allows you to have multiple events happening simultaneously, without pausing the program. This is done by measuring time (in milliseconds) passed since the program started.
- Then, with the use of intervals and continuously storing the time for last event, a simple algorithm can be made to have events happening at specific times without pausing the program.

Example

```
unsigned long previousMillis_1 = 0; //store time for first event
unsigned long previousMillis_2 = 0; //store time for second event

const long interval_1 = 1000; //interval for first event
const long interval_2 = 2000; //interval for second event

void setup(){}

void loop() {

//check time since program started, and store in "currentMillis"
unsigned long currentMillis = millis();

  //conditional that checks whether 1 second has passed since last event
  if (currentMillis - previousMillis_1 >= interval_1) {
      previousMillis_1 = millis();
      //execute a piece of code, every *1 second*
  }

  //conditional that checks whether 2 seconds have passed since last event
  if (currentMillis - previousMillis_2 >= interval_2) {
      previousMillis_2 = millis();
   //execute a piece of code, every *2 seconds*
  }
}
```

Functions – void example

```
int x;

void loop(){
   thisFunction(); //execute the function
}

void thisFunction() {
   x++; //increase x by 1 each time function is run.
}
```


Functions – int example

```
int value;

void setup(){}

void loop(){
   value = returnFunction();
}

int returnFunction() {
   int returnValue = 5 + 2;
   return returnValue;
}
```

Variables

- Variables can either be created locally or globally.
- Variables that are defined in the loop() are considered local, and variables defined at the top of your sketch are considered global.

64

Variables

```
//global variable
int sensorReading = x;
void setup(){}

void loop(){
   int sensorReading = x; //local variable
}
```


65

Data Types

- Some are:
- bool
- byte
- char
- double
- float
- int
- long
- short
- String

66

Variable Assignment

- int exampleNumber = 25;
- float exampleNumber = 22.2123002;
- String exampleSentence = "This is a string!";
- bool exampleSwitch = true; // true/false

67

Serial Communication

- Serial communication is essential to Arduino programming, as it is the easiest way to know what goes on, on your board.
- For this, we can use the Serial class.

- Serial.begin()

- Initializes serial communication between board & computer. This is defined in the void setup() function, where you also specify **baud rate** (speed of communication).

- Example
- void setup() {
- Serial.begin(9600);
- }

Serial Communication

- Prints data to the serial port, which can be viewed in the Arduino IDE Serial Monitor tool.
- void loop() {
- Serial.print();
- }

- Reads the incoming serial data.
- void loop() {
- int incomingByte = Serial.read();
- }

GPIO / Pin Management

- **GPIO / Pin Management**: Configuring, controlling and reading the state of a digital/analog pin on an Arduino.

pinMode

- pinMode() configures a digital pin to behave as an input or output.
- It is configured inside the void setup() function.

- Example:
- pinMode(pin, INPUT); //configures pin as an input
- pinMode(pin, OUTPUT); //configures pin as an output

digitalRead()

- Reads the state of a digital pin. Used to for example detect a button click.

- int state = digitalRead(pin);
- //store the state in the "state" variable

72

digitalWrite()

- Writes a high or low state to a digital pin. Used to switch on or off a component.

- digitalWrite(pin, HIGH); // writes a high (1) state to a pin (aka turn it on)
- digitalWrite(pin, LOW); // writes a low (0) state to a pin (aka turn it off)

73

analogRead()

- Reads the voltage of an analog pin, and returns a value between 0-1023 (10-bit resolution). Used to read analog components.

- sensorValue = analogRead(A1);
- //stores reading of A1 in "sensorValue" variable

74

analogWrite()

- Writes a value between 0-255 (8-bit resolution). Used for dimming lights or setting the speed of a motor. Also referred to as PWM, or Pulse Width Modulation.

- analogWrite(pin, value); //write a range between 0-255 to a specific pin

- PWM is only available on specific pins (marked with a "~" symbol).

75

Structure

- The structure of the Arduino API is based on C++ and can be considered the building blocks of a program.

Conditionals

- Conditionals are some of the most popular used elements in any program. In Arduino, a typical conditional consists of an if and else statement.

```
if(variable == true){
   //do something
}
else {
   //do something else
}
```

Loops / Iterations

- The for and while loops are commonly used in programs, to execute a block of code for a set number of times, or while a condition is met.
- A basic use of a while loop to execute a block of code while variable is true.

```
while (variable == true) {
    //do something
}
```


78

Loops / Iterations

- A basic use of a for loop is to execute a block of code a custom number of times (in this case, 10).

```
for (int x = 0; x < 10; x++) {
      //do something 10 times
  }
```


79

break

- To break out of a loop, you can use break. In the snippet below, if a condition is met (variable is true), we break out of the loop.

```
for (int x = 0; x <= 10; x++) {
    if(variable == true) {
        break;
    }
 }
```


80

Arithmetic Operators

- Arithmetic operators are used for addition, subtraction, multiplication, division and other mathematical calculations.

```
int x = 5;
int y = 2;

x + y; //result is 7
x * y; //result is 10
x - y; //result is 3
```


81

Comparison Operators

- Comparison operators are used for comparing one property to another, and are a key component of a conditional statement.

- There are several comparison operators:
- != //not equal to
- < //less than
- <= //less than or equal to
- == //equal to
- > //greater than
- >= //greater than or equal to
- Usage:
- if(value > 10) {
- //do something
- }

82

Boolean Operators

- Boolean operators (logical NOT !, AND && and OR ||) can for example be used for more advanced conditionals.

- To use the AND && operator:
- if(value > 10 && otherValue > 10){
- //do something if only if *both* conditions are met
- }
- To use the OR || operator:
- if(value > 10 || otherValue > 10){
- //do something if a one *or* the other condition is met
- }
- To use the NOT ! operator:
- if(!value){
- //do something if value is false (!)
- }

83

Compound Operators

- Compound operators consists of two operators, which are used to perform two operations in the same statement.

- x = 5;
- y = 2;

- x++; //increase by one, so x is now 6
- x--; //decrease by one, so x is now 4

- x += y; //x is now 7 (add and assign)
- x -= y; //x is now 3 (subtract and assign)
- x *= y; //x is now 10 (multiply and assign)

Some Arduino functions official page

- https://docs.arduino.cc/language-reference/

Home Task

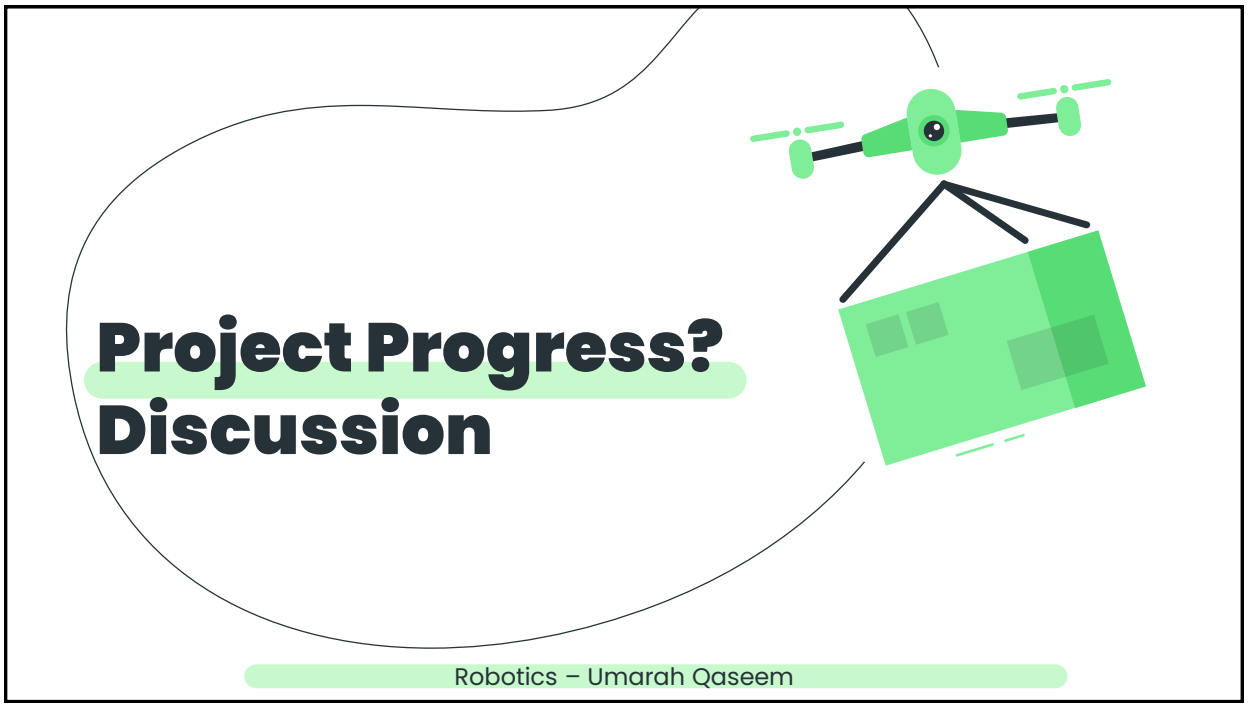
- Study PWM

86

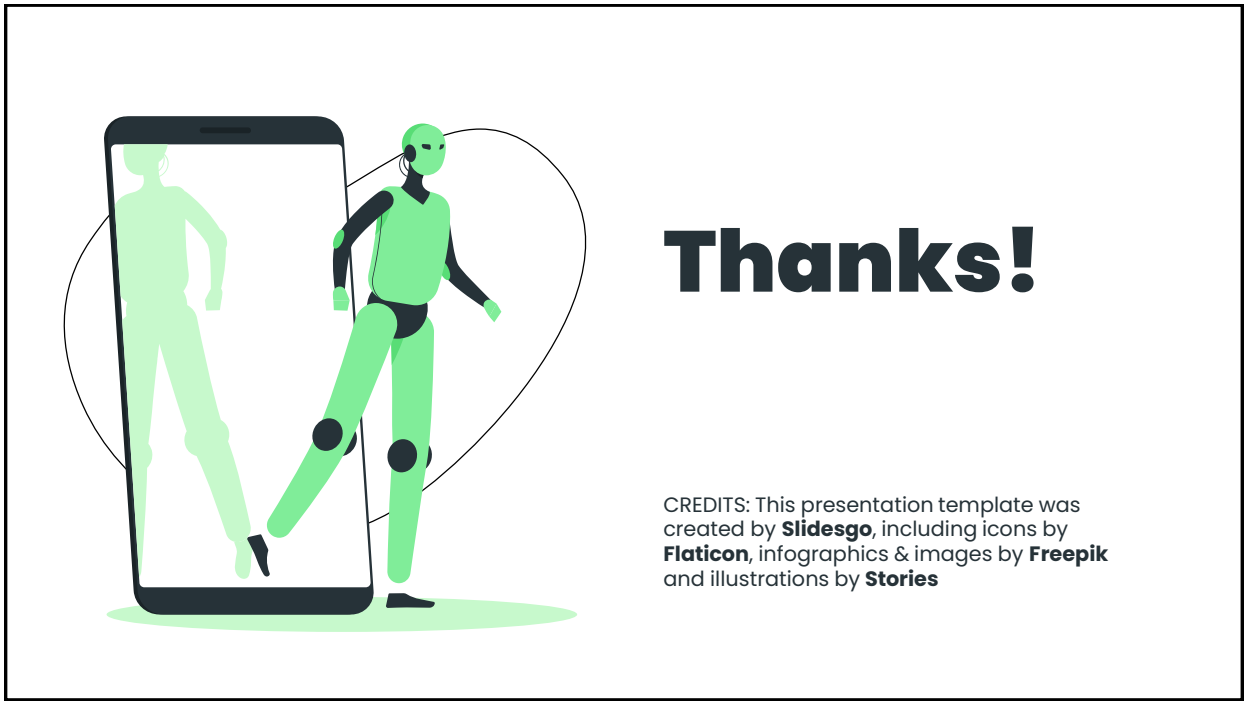
References

- Arduino Official Documentation

87


Project Progress?
Discussion
Robotics – Umarah Qaseem

88


Thanks!
CREDITS: This presentation template was created by Slidesgo, including icons by Flaticon, infographics & images by Freepik and illustrations by Stories

96